Room en el registro

Esta guía guarda cada envío del formulario en SQLite. Al pulsar **Registrarse** con los datos válidos, la app sigue escribiendo el nombre y el correo en DataStore y, además, inserta una fila. Debajo del formulario aparece la lista **Registros guardados**, con un botón para borrar cada una.

La foto sigue siendo opcional. La ruta `register_page` y el `NavHost` no se tocan. La cámara y `RegistroPreferences` se quedan como están.

1. Dependencias

En `app/build.gradle.kts`, el bloque `plugins` queda así:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    id("com.google.devtools.ksp") version "2.3.6"
}
```

KSP lee las anotaciones de Room y genera el código del DAO al compilar. La versión `2.3.6` es la que entiende el Kotlin integrado de Android Gradle Plugin 9. Desde KSP 2.3 el número ya no copia la versión de Kotlin. Una versión anterior, como `2.2.10-2.0.2`, registra los archivos generados por una vía que este proyecto rechaza.

Dentro de `dependencies`, agrega esto y sincroniza Gradle:

```
implementation("androidx.room:room-runtime:2.8.5")
implementation("androidx.room:room-ktx:2.8.5")
ksp("androidx.room:room-compiler:2.8.5")
```

- `room-runtime` abre el archivo SQLite y ejecuta las consultas.
- `room-ktx` permite funciones suspendidas y `Flow` en el DAO.
- `room-compiler` es el que KSP ejecuta para generar la implementación.

Sin sincronizar, las anotaciones `@Entity`, `@Dao` y `@Database` no se reconocen.

2. La fila

Crea `app/src/main/java/cl/duoc/myapplication/data/Registro.kt`:

```kotlin
package cl.duoc.myapplication.data

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "registros")
data class Registro(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val nombre: String,
    val correo: String,
    val fotoUri: String,
    val creadoEn: Long
)
```

`@Entity` marca la clase como una tabla. `tableName` es el nombre de esa tabla dentro del archivo. Cada propiedad es una columna.

`id` es la clave. `autoGenerate = true` hace que SQLite ponga el número al insertar. El `0` inicial le dice a Room que esa fila todavía no tiene clave.

`nombre` y `correo` son el texto del envío. `fotoUri` es la dirección de la foto, o `""` si no había. `creadoEn` es la hora del envío en milisegundos. El mismo correo puede repetirse: cada pulsación de **Registrarse** es otra fila.

3. Las consultas

En el mismo paquete, crea `RegistroDao.kt`:

```kotlin
package cl.duoc.myapplication.data

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query
import kotlinx.coroutines.flow.Flow

@Dao
interface RegistroDao {
    @Insert
    suspend fun insertar(registro: Registro)

    @Query("SELECT * FROM registros ORDER BY creadoEn DESC")
    fun observar(): Flow<List<Registro>>

    @Query("DELETE FROM registros WHERE id = :id")
    suspend fun borrar(id: Long)
}
```

`@Dao` es el acceso a la tabla. Room escribe la implementación.

`insertar` agrega una fila. Es `suspend` porque la escritura no va en el hilo que dibuja la pantalla: hay que llamarla desde una corrutina.

`observar` no es `suspend`. Devuelve un `Flow`, una secuencia que emite la lista otra vez cada vez que la tabla cambia. `ORDER BY creadoEn DESC` deja el envío más nuevo arriba. `:id` en el `DELETE` es el parámetro de `borrar`.

4. El archivo de la base

Crea `AppDatabase.kt` en el mismo paquete:

```
package cl.duoc.myapplication.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Registro::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {
    abstract fun registroDao(): RegistroDao

    companion object {
        @Volatile
        private var instancia: AppDatabase? = null

        fun obtener(context: Context): AppDatabase {
            return instancia ?: synchronized(this) {
                instancia ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "registro.db"
                ).build().also { instancia = it }
            }
        }
    }
}
```

`@Database` junta las entidades. `version = 1` es la primera forma de la tabla. `exportSchema = false` deja el esquema en estas anotaciones, sin un JSON de historial.

`registroDao()` es la puerta a las consultas. Room genera el cuerpo.

`obtener` abre `registro.db` una sola vez por proceso. `@Volatile` hace visible esa instancia entre hilos. `synchronized` evita que dos llamadas al mismo tiempo creen dos bases. `applicationContext` ata el archivo a la aplicación, no a una pantalla.

5. El ViewModel

`RegistroViewModel` ya escribe el borrador en DataStore. Ahora también inserta la fila y publica la lista.

Imports nuevos:

```
import cl.duoc.myapplication.data.AppDatabase
import cl.duoc.myapplication.data.Registro
```

`collect` es un método de `Flow`. No lleva un import propio.

Junto a `estado`, la lista:

```
private val dao = AppDatabase.obtener(application).registroDao()
private val _registros = MutableStateFlow<List<Registro>>(emptyList())
val registros: StateFlow<List<Registro>> = _registros.asStateFlow()
```

`dao` es el acceso a la tabla. `_registros` parte vacía. `registros` es la versión de solo lectura que mira la pantalla.

En `init`, deja la lectura de DataStore y agrega un segundo `launch` que escucha la tabla:

```
init {
    viewModelScope.launch {
        val guardado = preferences.leer()
        _estado.update {
            it.copy(nombre = guardado.nombre, correo = guardado.correo)
        }
    }
    viewModelScope.launch {
        dao.observar().collect { lista ->
            _registros.value = lista
        }
    }
}
```

`collect` se queda esperando. Cada vez que Room emite la lista, `_registros` se actualiza y Compose vuelve a dibujar.

`guardar()` inserta después de escribir el borrador. El nombre, el correo y la foto son los que había al pulsar el botón, guardados en `actual`:

```
fun guardar() {
    val actual = _estado.value
    viewModelScope.launch {
        _estado.update { it.copy(guardando = true) }
        delay(800)
        preferences.guardar(actual.nombre, actual.correo)
        dao.insertar(
            Registro(
                nombre = actual.nombre,
                correo = actual.correo,
                fotoUri = actual.fotoUri,
                creadoEn = System.currentTimeMillis()
            )
        )
        _estado.update { it.copy(guardando = false, guardado = true) }
    }
}
```

`id` no se pasa: vale `0` y SQLite asigna el número. `fotoUri` puede ir vacío. La fila se inserta igual.

Debajo de `onFotoTomada`, el borrado:

```
fun borrarRegistro(id: Long) {
    viewModelScope.launch {
        dao.borrar(id)
    }
}
```

`borrar` quita esa fila. `observar()` emite la lista sin ella. DataStore y el archivo de la foto no se tocan.

6. La lista en la pantalla

En `RegisterScreen`, junto a `estado`, observa la lista:

```
val registros by viewModel.registros.collectAsState()
```

`collectAsState` ya está importado: es el mismo que usa `estado`.

Debajo del `AnimatedVisibility` de “Datos guardados en el teléfono”, y antes del botón **Volver**, agrega:

```
HorizontalDivider(modifier = Modifier.padding(vertical = 16.dp))

Text("Registros guardados")

if (registros.isEmpty()) {
    Text("Todavía no hay registros")
}

registros.forEach { registro ->
    Text(registro.nombre)
    Text(registro.correo)
    Text(formatoFecha(registro.creadoEn))
    Button(onClick = { viewModel.borrarRegistro(registro.id) }) {
        Text("Borrar")
    }
}
```

La columna ya se desplaza, así que la lista puede crecer sin tapar **Volver**.

Al final del archivo, junto a `cargarFoto`, el texto de la fecha:

```
private fun formatoFecha(millis: Long): String {
    val formato = SimpleDateFormat("dd/MM/yyyy HH:mm", Locale.getDefault())
    return formato.format(Date(millis))
}
```

Imports:

```
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale
```

`creadoEn` es un número de milisegundos. `SimpleDateFormat` lo pasa a día, mes, año y hora del teléfono. Cada llamada crea su propio formato, así que no se comparte entre hilos.

Qué probar

1. Sincroniza Gradle y corre la app. En registro, debajo del formulario, debe decir “Todavía no hay registros”.
2. Pulsa **Registrarse** con el formulario vacío. No aparece ninguna fila.
3. Completa nombre, correo y términos, y pulsa **Registrarse**. Al terminar el guardado, la lista muestra esa fila con la fecha.
4. Envía otro registro, aunque repita el correo. La fila nueva queda arriba.
5. Pulsa **Borrar** en una fila. Esa fila sale de la lista. El nombre del formulario sigue en los campos, porque el borrador vive en DataStore.
6. Cierra la app y ábrela de nuevo. La lista sigue. El borrador del formulario también.